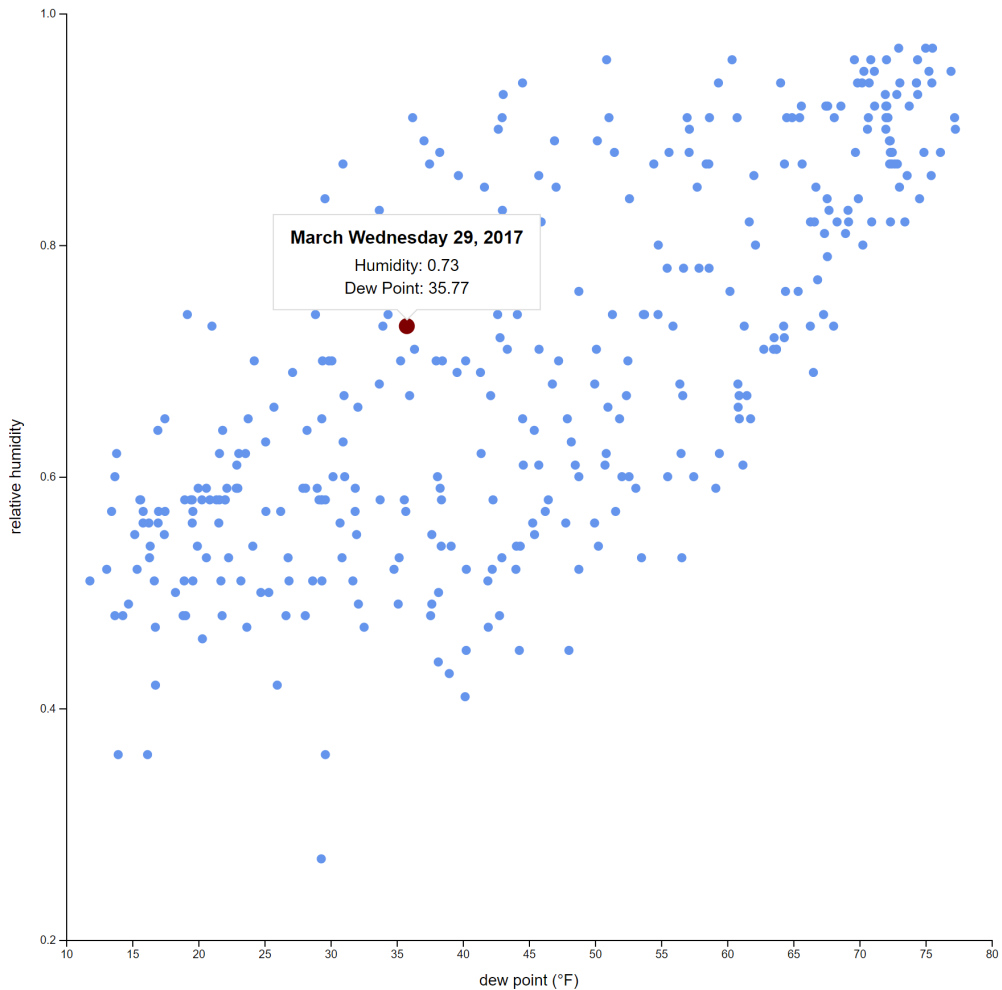




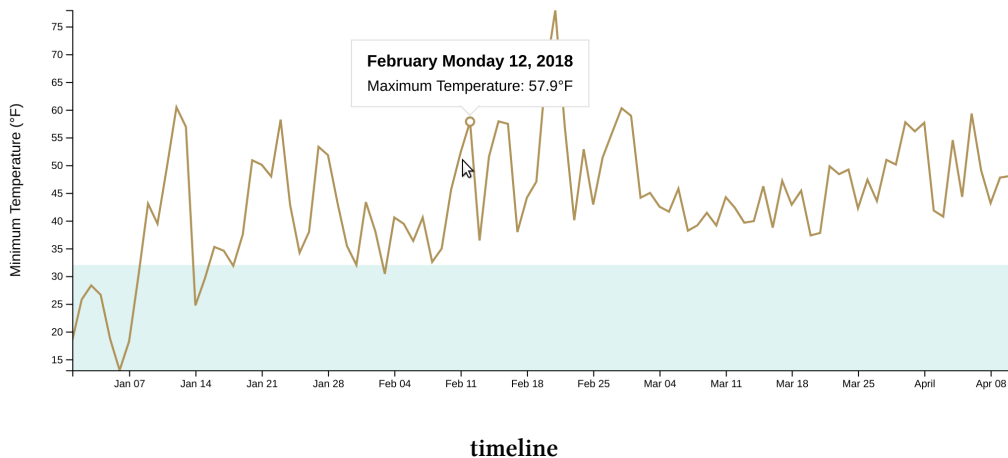
scatter plot

Making a tooltip for our scatter plot was trickier than expected, but we saw how important encouraging interaction can be. When our hover targets were small, it felt like work to get more information about a specific point. But now that we're using voronoi cells, interacting with our chart is almost fun!

Line chart

Let's go through one last example for adding tooltips. So far, we've added tooltips to individual elements (bars, circles, and paths). Adding a tooltip to a timeline is a bit different. Let's dig in by navigating to `/code/05-interactions/4-line/draft/` in the browser and `/code/05-interactions/4-line/draft/line.js` in your text editor.

In this section, we're aiming to add a tooltip to our line chart like this:



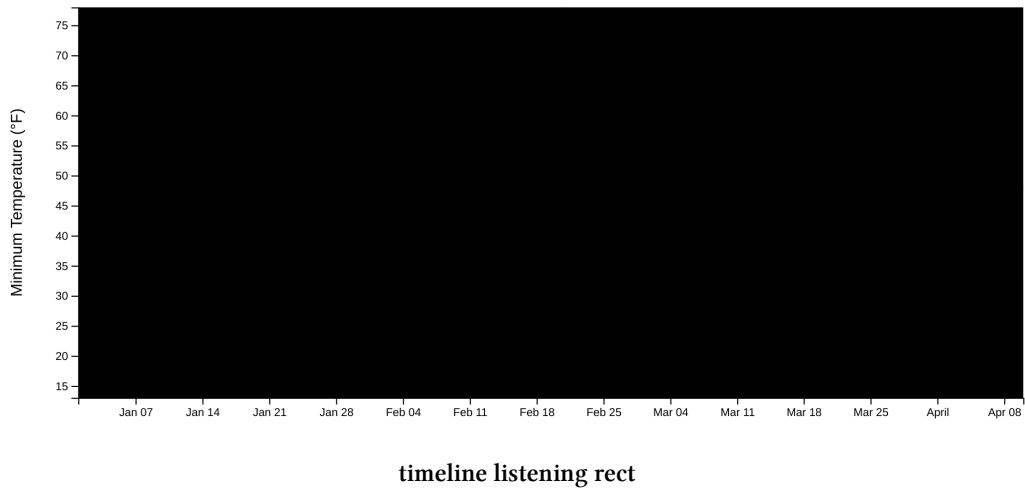
Instead of catching hover events for individual elements, we want to display a tooltip whenever a user is hovering **anywhere on the chart**. Therefore, we'll want an element that spans our entire **bounds**.

To start coding up our **Set up interactions** step, let's create a `<rect>` that covers our **bounds** and add our mouse event listeners to it. This time we'll want to listen for `mousemove` events instead of `mouseenter` events because we'll want to update the tooltip's position when a reader moves their mouse around the chart.

Note that we don't need to define our `<rect>`'s `x` or `y` attributes because they both default to `0`.

```
const listeningRect = bounds.append("rect")
    .attr("class", "listening-rect")
    .attr("width", dimensions.boundedWidth)
    .attr("height", dimensions.boundedHeight)
    .on("mousemove", onMouseMove)
    .on("mouseleave", onMouseLeave)
```

Perfect! We can see that our `listeningRect`, defaulted to a black fill, covers our entire bounds.



Let's add a rule to `styles.css` so we can see our chart again.

```
.listening-rect {
    fill: transparent;
}
```

Great! Now we can set up our `tooltip` variable and `onMouseMove` and `onMouseLeave()` functions (back in our `line.js` file).

```
const tooltip = d3.select("#tooltip")
function onMouseMove() {
}

function onMouseLeave() {
}
```

Let's start fleshing out `onMouseMove` — how will we know the location on our line that we are hovering over? The passed parameters we used previously (`datum`, `index`, and `nodes`) won't be helpful here, and `this` will just point us at the listener `rect` element.

When an event listener is invoked, the **d3-selection** library sets a global `d3.event`. `d3.event` will refer to the currently triggered event and will be reset when the event listener is done. During the event listener handler, we also get access to a `d3.mouse()` function which will return the `x`, `y` coordinates of the mouse event, relative to a specified container.

Let's see what that would look like in action and pass our listener container to `d3.mouse()`.

```
function onMouseMove() {
  const mousePosition = d3.mouse(this)
  console.log(mousePosition)
```

Now we can see our mouse position as an `[x,y]` array when we move our mouse around the chart.

```
► (2) [220.99826049804688, 103.00520324707031]
```

mouse coordinates

Test it out — what do the numbers look like when you hover over the top left of the chart? What about the bottom right?

There was a [bug that will be fixed in the next version of Firefox \(68\)](#)^a that fails to properly offset the `d3.mouse()`'s reported mouse coordinates, disregarding any CSS transform: `translate()` properties on ancestor elements within an `<svg>` element.

Until most people have updated their Firefox to more recent versions, make sure that you set the `transform` HTML attribute instead of setting a CSS transform property if possible (for example, when shifting your **bounds**), if you intend on grabbing the mouse position.

^ahttps://bugzilla.mozilla.org/show_bug.cgi?id=972041

Great, but in order to show the tooltip next to an actual data point, we need to know which point we're closest to. First, we'll need to figure out what date we're hovering over — how do we convert an `x` position into a date? So far, we've only used our scales to convert from the data space (in this case, JavaScript date objects) to the pixel space.

Thankfully, d3 scales make this very simple! We can use the same `xScale()` we've used previously, which has an `.invert()` method. `.invert()` will convert our units backwards, from the range to the domain.

Let's pass the `x` position of our mouse (`mousePosition[0]`) to the `.invert()` method of our `xScale()`.

```
const mousePosition = d3.mouse(this)
const hoveredDate = xScale.invert(mousePosition[0])
```

Okay great, now know what date we're hovering over — let's figure out how to find the closest data point.

d3.scan()

If you ever need to know where a variable will fit in a sorted list, `d3.scan()`³⁶ can help you out. `d3.scan()` requires two parameters:

³⁶<https://github.com/d3/d3-array#scan>

1. an array (in this case, our dataset), and
2. an optional comparator function.

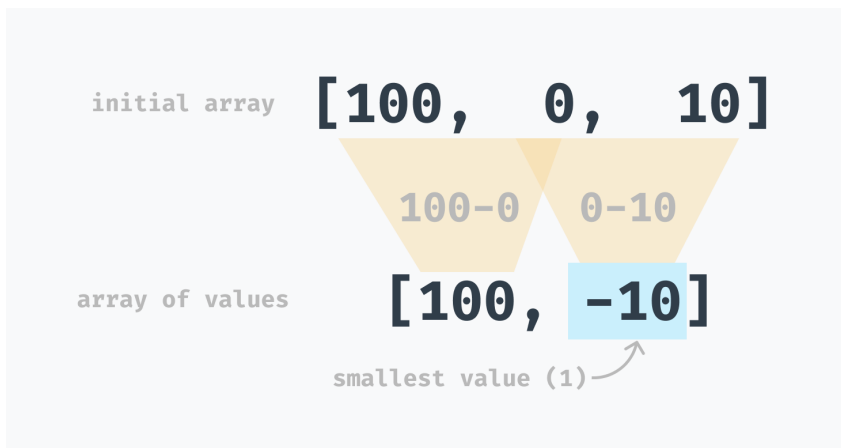
The comparator function will take two adjacent items in the passed array and return a numerical value. `d3.scan()` will take those returned values and return the *index* of the *smallest value*.

The general d3 bundle uses the 1.0 version of d3-array. If you're importing a newer version of d3-array, versions 2.2.0 and up have replaced `d3.scan()` with `d3.leastIndex()`^a.

^a<https://github.com/d3/d3-array#leastIndex>

Let's look at a few examples to get that description to *click*:

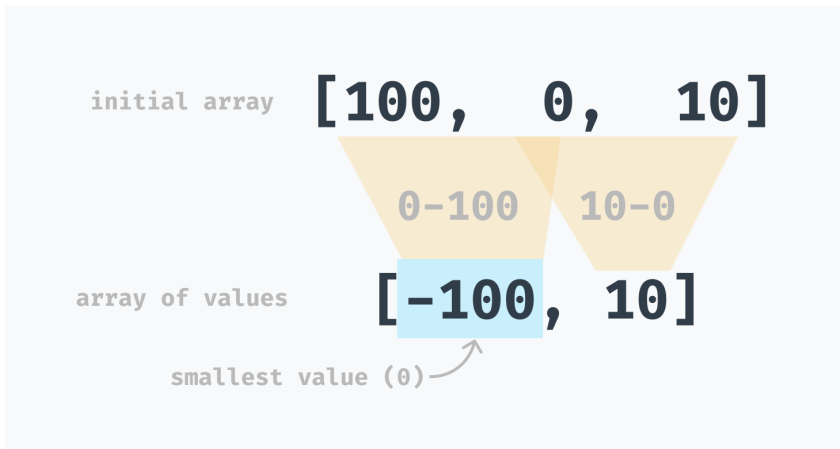
`d3.scan([100, 0, 10], (a,b) => a - b)` would create an array of values that looks like `[100, -10]`.



`d3.scan()` example

This expression would then return 1 because the second item in the array of values is the smallest (remember, the second item is referred to as 1 when we're looking at zero-indexed indices).

`d3.scan([100, 0, 10], (a,b) => b - a)` would create the array `[-100, 10]`



d3.scan() example

This expression would then return 0, because the first item of the array of values is the smallest.

Let's try it out — we'll first create a function to find the distance between the hovered point and a datapoint. We don't care if the point is before or after the hovered date, so we'll use `Math.abs()` to convert that distance to an absolute distance.

```
const getDistanceFromHoveredDate = d => Math.abs(
  xAccessor(d) - hoveredDate
)
```

Then we can use that function to compare the two data points in our `d3.scan()` comparator function. This will create an array of distances from the hovered point, and we'll get the index of the closest data point to our hovered date.

```
const closestIndex = d3.scan(dataset, (a, b) => (
  getDistanceFromHoveredDate(a) - getDistanceFromHoveredDate(b)
))
```

Next, we need to grab the data point at that index.

```
const closestDataPoint = dataset[closestIndex]
```

Let's `console.table(closestDataPoint)` to make sure we're grabbing the right value.

(index)	Value
apparentTemperatureMinTi...	1483380000
precipType	"rain"
temperatureMin	37.22
summary	"Clear throughout the da...
dewPoint	35.15
apparentTemperatureMax	40.43
temperatureMax	45.34
temperatureMaxTime	1483434000
windBearing	223
moonPhase	0.16
sunsetTime	1483433080
pressure	1023.9
uvIndexTime	1483423200
apparentTemperatureMin	29.68
date	"2017-01-03"
icon	"clear-day"
apparentTemperatureMaxTi...	1483423200
humidity	0.78
windSpeed	7.63
time	1483372800
uvIndex	1
sunriseTime	1483398430
temperatureMinTime	1483380000

► Object

console table example

When we move our mouse to the left of our chart, we should see dates close to the beginning of our dataset, which increase as we move right.

Perfect! Now let's grab the closest x and y values using our accessor functions — these will come in handy when we're updating our tooltip.

```
const closestXValue = xAccessor(closestDataPoint)
```

```
const closestYValue = yAccessor(closestDataPoint)
```

We can use our `closestXValue` to set the date in our tooltip. Let's also format it nicely using `d3.timeFormat()` with the same specifier string we used for our scatter plot.

```
const formatDate = d3.timeFormat("%B %A %-d, %Y")
tooltip.select("#date")
  .text(formatDate(closestXValue))
```

Next up, we can set the temperature value in our tooltip — this time our formatter string will also add a °F suffix to clarify.

```
const formatTemperature = d => `${d3.format(".1f")(d)}°F`
tooltip.select("#temperature")
  .html(formatTemperature(closestYValue))
```

Note that we want to use `.html()` here, to ensure that our degrees symbol will be parsed correctly.

Lastly, we'll want to grab the x and y position of our closest point, shift our tooltip, and hide/show our tooltip appropriately. This should look like what we've done in the past two sections.

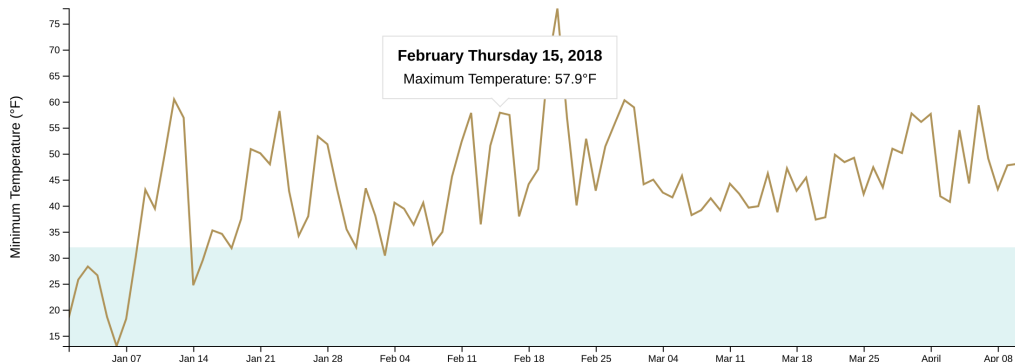
```
const x = xScale(closestXValue)
  + dimensions.margin.left
const y = yScale(closestYValue)
  + dimensions.margin.top

tooltip.style("transform", `translate(`
  + `calc( -50% + ${x}px),`
  + `calc(-100% + ${y}px)`
  + `)``)

tooltip.style("opacity", 1)
}

function onMouseLeave() {
  tooltip.style("opacity", 0)
}
```

Wonderful! When we refresh our webpage, we can see a tooltip that will match the horizontal position of our cursor, while sitting just above our line.



timeline with tooltip

Extra credit

You may notice an issue that we had before with our scatter plot — it's not immediately clear what point we're hovering over. Let's solve this by positioning a `<circle>` over the spot we're hovering — this should make the interaction clearer and the dataset more tangible.

First, we need to create our circle element — let's draw it right after we create our tooltip variable. We'll hide it with an `opacity` of 0 to start.

```
const tooltip = d3.select("#tooltip")
const tooltipCircle = bounds.append("circle")
  .attr("r", 4)
  .attr("stroke", "#af9358")
  .attr("fill", "white")
  .attr("stroke-width", 2)
  .style("opacity", 0)
```

Now, right after we position our tooltip in `onMouseEnter()`, we can also position our `tooltipCircle` and give it an `opacity` of 1.

```

tooltipCircle
  .attr("cx", xScale(closestXValue))
  .attr("cy", yScale(closestYValue))
  .style("opacity", 1)
}

```

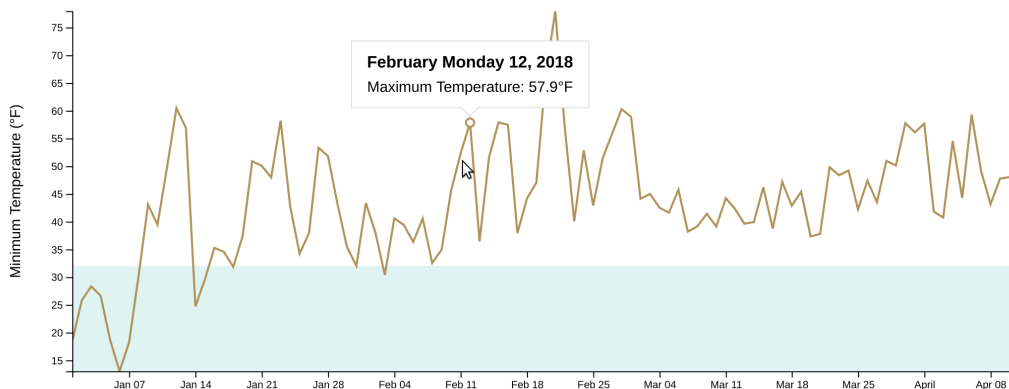
Lastly, we'll hide it in `onMouseLeave()` after we hide our tooltip.

```

function onMouseLeave() {
  tooltip.style("opacity", 0)
  tooltipCircle.style("opacity", 0)
}

```

Voila! Now we should see a circle under our tooltip, right over the “hovered” point.



finished timeline

Give it a spin and feel out the difference. Putting yourself in the user's shoes, you can see how highlighting the hovered data point makes the data feel more tangible.

Making a map

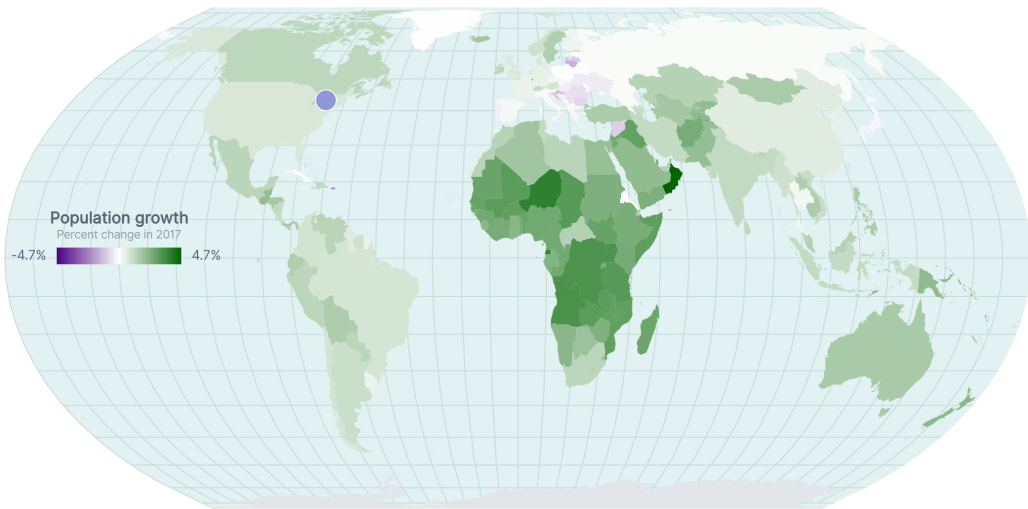
Let's take a step back from our weather data for a chapter and replace it with a dataset of population growth per country. We could visualize our country data as a bar chart, which would be great for finding the countries with the highest population growth. Instead, let's visualize this data using a **choropleth map**, which is a map with shaded areas representing a metric's values.

Note that a map isn't always the best way to visualize geographic data — that depends on the goal of the visualization (and the geographic literacy of your audience). One of the main benefit of maps is that they mimic our knowledge of the world — name a country and the first thing that comes to mind is where it's located in the world. This knowledge helps to navigate the visualization — there are almost 200 countries, but we can almost instantaneously find our own.

Maps are also uniquely good at answering several questions, such as:

- Do neighboring countries tend to have the same amount of population growth?
- Are there patterns across continents?
- Do geographically larger countries tend to have more or less population growth?

In this chapter, we'll build the following map:



Finished map

Digging in

The first step to making a map is to decide what it will be composed of. Will it have streets, mountains, cities, or lakes? We want to visualize data at the country level, so we need to make a map that consists of every country. Hmm, how do we find the shape and location of each country?

To create our map, we'll use the [d3-geo](https://github.com/d3/d3-geo)³⁷ module which accepts GeoJSON data.

What is GeoJSON?

GeoJSON is a format used to represent geographic structures (a geometry, a feature, or a collection of features). A GeoJSON object can contain features of the following types: Point, MultiPoint, LineString, MultiLineString, Polygon, MultiPolygon, GeometryCollection, Feature, or FeatureCollection. If you're curious, check out [the spec](https://tools.ietf.org/html/rfc7946)³⁸ for more information.

³⁷<https://github.com/d3/d3-geo>

³⁸<https://tools.ietf.org/html/rfc7946>

There are many sources of GeoJSON files. We'll use [Natural Earth](https://www.naturalearthdata.com/)³⁹, which is a large collection of public domain map data of various features, locations, and granularities. We'll use the **Admin 0 - Countries** dataset, downloadable [here](https://www.naturalearthdata.com/downloads/50m-cultural-vectors/50m-admin-0-countries-2/)⁴⁰.

Feel free to either follow along and generate the GeoJSON file yourself or read through this section as an observer. Either way, you'll get an idea of how to create your own GeoJSON file for your custom maps. This resource will still be here when you want to dive deeper.

When we download the countries from **Natural Earth**, we'll get a zip file of various formats.



```
ne_50m_admin_0_countries.cpg
ne_50m_admin_0_countries.dbf
ne_50m_admin_0_countries.prj
ne_50m_admin_0_countries.README.html
ne_50m_admin_0_countries.shp
ne_50m_admin_0_countries.shx
ne_50m_admin_0_countries.VERSION.txt
```

Natural Earth zip files

We're interested in the **shapefile**: `ne_50m_admin_0_countries.shp`. A **shapefile** is another format for representing geographic data. Let's extract it into our `/code/06-making-a-map/` folder to turn it into a GeoJSON file.

Thankfully, there are many tools for converting **shapefiles** into **GeoJSON** objects. We'll use the **Geographic Data Abstraction Library (GDAL)**⁴¹ - check out [this page](https://www.gdal.org/)⁴² to download it, or if you're on a Mac, run `(brew install gdal)`.

Once we have **GDAL** installed, we can convert our `ne_50m_admin_0_countries.shp` **shapefile** into a JSON file containing a **GeoJSON** object. The following command will do that and throw the output into a file called `world-geojson2.json`. (The

³⁹<https://www.naturalearthdata.com/>

⁴⁰<https://www.naturalearthdata.com/downloads/50m-cultural-vectors/50m-admin-0-countries-2/>

⁴¹<https://www.gdal.org/>

⁴²<http://trac.osgeo.org/gdal/wiki/DownloadingGdalBinaries>

at a `FeatureCollection` named "ne_50m_admin_0_countries"), except for the features array.

Let's expand the features array and look inside.

```
▼ {type: "FeatureCollection", name: "ne_50m_admin_0_countries", crs: {...}, features: Array(241)} ⓘ
  ► crs: {type: "name", properties: {...}}
  ▼ features: Array(241)
    ▼ [0 ... 99]
      ► 0: {type: "Feature", properties: {...}, geometry: {...}}
      ▼ 1:
        ► geometry: {type: "Polygon", coordinates: Array(1)}
        ► properties: {featurecla: "Admin-0 country", scalerank: 1, LABELRANK: 3, SOVEREIGNT: "Zambia", S...
          type: "Feature"
          ► __proto__: Object
        ► 2: {type: "Feature", properties: {...}, geometry: {...}}
        ► 3: {type: "Feature", properties: {...}, geometry: {...}}
        ► 4: {type: "Feature", properties: {...}, geometry: {...}}
        ► 5: {type: "Feature", properties: {...}, geometry: {...}}
        ► 6: {type: "Feature", properties: {...}, geometry: {...}}
        ...
```

GeoJSON data, features

Each feature has a geometry object and a properties object. The properties object contains information about the feature — we can see that each of these features represents one country, based on the information in here.

```
▼ {type: "FeatureCollection", name: "ne_50m_admin_0_countries", crs: {...}, features: Array(241)} ⓘ
  ► crs: {type: "name", properties: {...}}
  ▼ features: Array(241)
    ▼ [0 ... 99]
      ► 0: {type: "Feature", properties: {...}, geometry: {...}}
      ▼ 1:
        ► geometry: {type: "Polygon", coordinates: Array(1)}
        ▼ properties:
          ABBREV: "Zambia"
          ABBREV_LEN: 6
          ADM0_A3: "ZMB"
          ADM0_A3_IS: "ZMB"
          ADM0_A3_UN: -99
          ADM0_A3_US: "ZMB"
          ADM0_A3_WB: -99
          ADM0_DIF: 0
          ADMIN: "Zambia"
          BRK_A3: "ZMB"
          BRK_DIFF: 0
          BRK_GROUP: null
          BRK_NAME: "Zambia"
          CONTINENT: "Africa"
          ECONOMY: "7. Least developed region"
          FIPS_10 : "ZA"
          FORMAL_EN: "Republic of Zambia"
          FORMAL_FR: null
```

GeoJSON data, geometry

For example, this feature has a NAME of "Zambia". If we dig into the geometry of

this feature, we'll find an array of `[latitude, longitude]` coordinates.

```

▼ {type: "FeatureCollection", name: "ne_50m_admin_0_countries", crs: {...}, features: Array(241)}
  ► crs: {type: "name", properties: {...}}
  ▼ features: Array(241)
    ▼ [0 ... 99]
      ► 0: {type: "Feature", properties: {...}, geometry: {...}}
      ▼ 1:
        ▼ geometry:
          ▼ coordinates: Array(1)
            ▼ 0: Array(377)
              ▼ [0 ... 99]
                ► 0: (2) [30.39609375, -15.64306640625]
                ► 1: (2) [30.25068359375001, -15.643457031250009]
                ► 2: (2) [29.994921875000017, -15.64404296875]
                ► 3: (2) [29.729589843750006, -15.644628906250006]
                ► 4: (2) [29.4873046875, -15.69677734375]
                ► 5: (2) [29.287890625000017, -15.776464843750006]
                ► 6: (2) [29.050585937500017, -15.901171875]
                ► 7: (2) [28.973046875000023, -15.950097656250009]
                ► 8: (2) [28.9130859375, -15.98779296875]

```

GeoJSON data, geometry

Latitude and **longitude** are part of a common geographic coordinate system:

latitude represents the North-South position of a point on Earth as the angle above (up to 90°) or below (down to -90°) the equator (0°).

longitude represents the East-West position of a point on Earth as an angle between -180° and 180°.

Fun fact: historically, geographers used a nation's capital as the 0° reference for **longitude**. In 1884, the International Meridian Conference decided on the universal **longitude** that we use today.

Next, we want to create our accessor functions. We'll need to access our country ID to find the metric value in our population growth dataset (which we'll look at later in this chapter). We'll also want our hovered country's name to display in a tooltip.

We can look in a sample feature to find the relevant keys.

code/06-making-a-map/completed/draw-map.js

```
8  const countryNameAccessor = d => d.properties["NAME"]  
9  const countryIdAccessor = d => d.properties["ADM0_A3_IS"]
```

Our dataset

Let's dig in to the code! We'll be working in the `/code/06-making-a-map/draft/` folder, starting with the `draw-map.js` file.

Let's grab our dataset that shows population growth — it's located in the `/code/06-making-a-map/data_bank_data.csv` file.

This data was pulled from [The World Bank](https://databank.worldbank.org/data/source/world-development-indicators#)^a — feel free to download a more recent dataset if you'd like. The population growth metric is the percent change of total population in 2017.

^a<https://databank.worldbank.org/data/source/world-development-indicators#>

Note that this is a CSV (comma-separated values) instead of a JSON file. No worries! This is just a different format of storing information — it ends up being much smaller than a JSON file because it only mentions each data point's properties once, similar to a table's header. Open the file up and take a peek!

To load data from a CSV file, we can make a simple tweak. Instead of using `d3.json()`, we can use `d3.csv()`.

code/06-making-a-map/completed/draw-map.js

```
6  const dataset = await d3.csv("../data_bank_data.csv")
```

Put this line of code on line 4, right after we load `countryShapes` to keep the file loading in one place.

Easy peasy! Let's log the result to the console and take a look (`console.log(dataset)`).

[illegible]

dataset

This data structure is a bit messy — a great chance to practice with a more real-world scenario! We can see that our dataset is an array which lists each country multiple times, each time with a different metric, named under the `Series Name` key.

We're only interested in the "Population growth (annual %)" metric. Let's define that in a variable name.

code/06-making-a-map/completed/draw-map.js

10

```
const metric = "Population growth (annual %)"
```

Later on, once you're more comfortable, play around with changing this metric value and visualize the other metrics.

We'll want an easy way to look up the population growth number using a country name. We could find a matching item in our `dataset` array, but that could be expensive. Instead, let's create an object with **country ids** as **keys** and the population growth amount as **values**:

```
{  
  AFG: 2.49078956147291,  
  ALB: -0.0919722937442495,  
  // ...  
}
```

To start, we'll initialize an empty object:

code/06-making-a-map/completed/draw-map.js

```
14  let metricDataByCountry = {}
```

Then we'll go over each item in our `dataset` array. If the item's "Series Name" doesn't match our `metric`, we won't do anything. If it does match, we'll add a new **value** to our `metricDataByCountry` object: the **key** is the item's "Country Code" and the **value** is the item's "2017 [YR2017]" number.

Note that these values are stored as **Strings** — we'll convert the **value** to a number by prepending `+`, and default to `0` if the **value** doesn't exist.

code/06-making-a-map/completed/draw-map.js

```
15  dataset.forEach(d => {  
16    if (d["Series Name"] !== metric) return  
17    metricDataByCountry[d["Country Code"]] = +d["2017 [YR2017]"] || 0  
18  })
```

That's a messy operation, isn't it? This is part of the reason to deal with the data at the top of our chart-making code. We'll polish our dataset into a cleaner format, separating handling our data idiosyncrasies from actually drawing the chart.

Create chart dimensions

Let's move on! Next up, we'll define our chart dimensions. We know that we want our chart to be 90% as wide as our window and we want a small margin on every side of our map.

Let's hold off on setting the height for now.

code/06-making-a-map/completed/draw-map.js

```
22  let dimensions = {
23    width: window.innerWidth * 0.9,
24    margin: {
25      top: 10,
26      right: 10,
27      bottom: 10,
28      left: 10,
29    },
30  }
31  dimensions.boundedWidth = dimensions.width
32    - dimensions.margin.left
33    - dimensions.margin.right
```

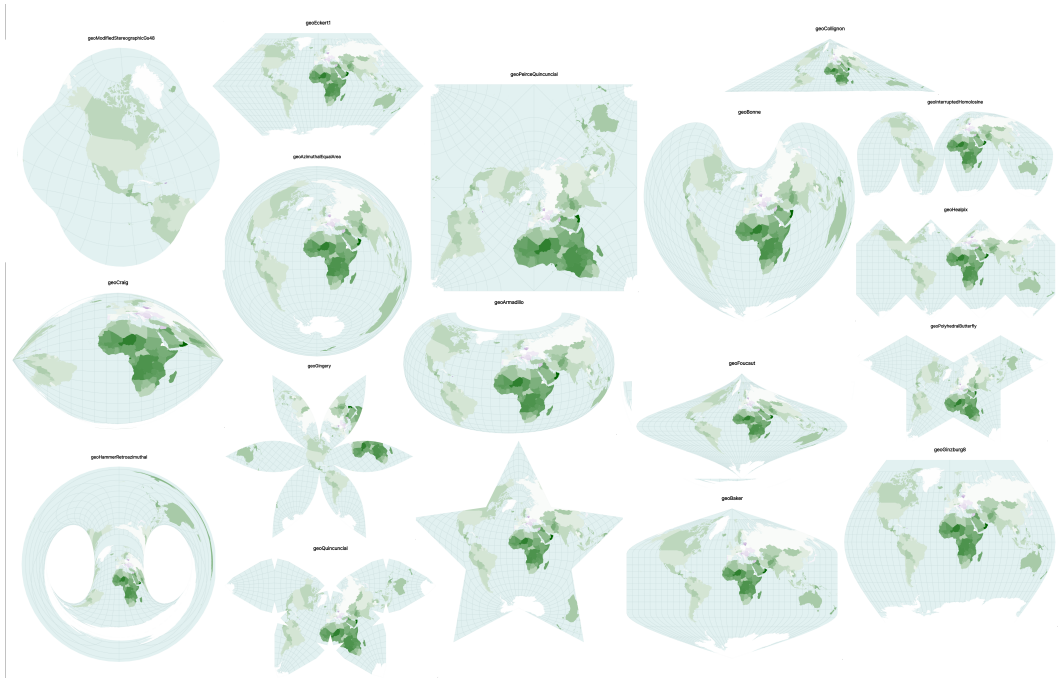
How tall does our map need to be? That will depend on our map's **projection**.

What is a projection?

Despite what flat-earthers might say, the Earth is a sphere. When representing it on a 2D screen, we'll need to create some rules — these rules are the **projection**.

Imagine peeling an orange and turning the skin into a flat sheet — even with slicing it in various places, it's impossible to do perfectly. **Projections** will use a combination of distortion (stretching parts of the map) and slicing to approximate the Earth's actual shape.

There are *many* projections out there. **d3-geo**⁴³ has 15 projections built in, but many more in a d3 module that's not included in the core build: **d3-geo-projection**⁴⁴. Play around with the different options at <http://localhost:8080/06-making-a-map/completed-projections>⁴⁵.



projections, galore!

Which projection should I use?

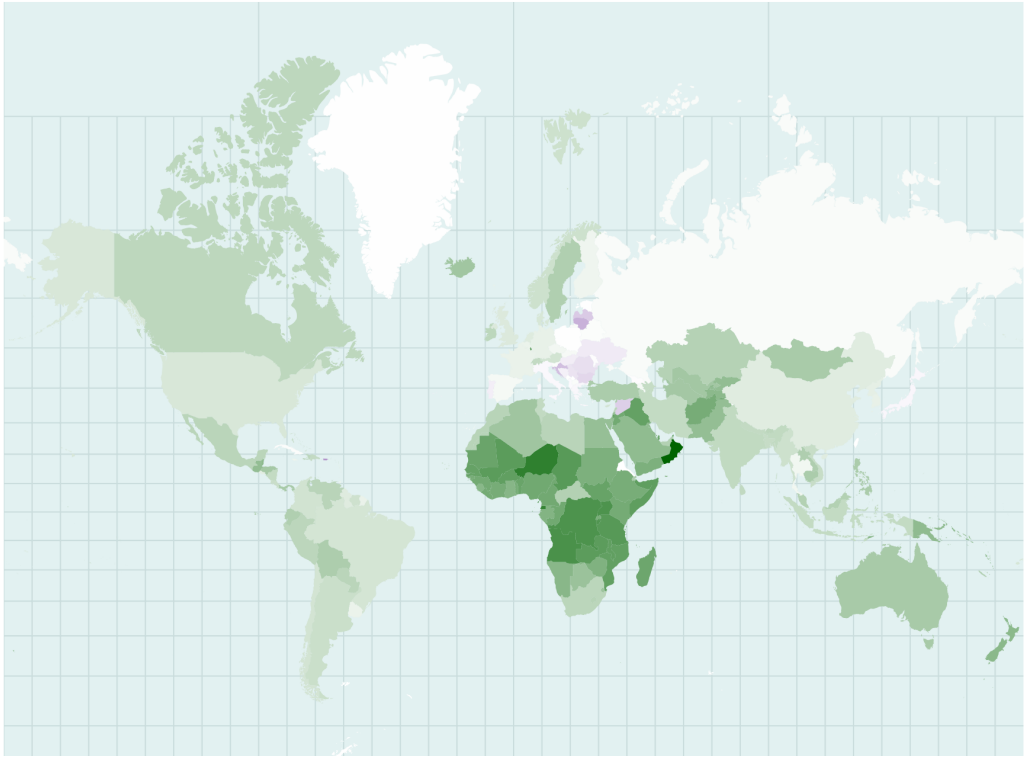
The projection you might be most used to is the **Mercator** projection.

⁴³<https://github.com/d3/d3-geo>

⁴⁴<https://github.com/d3/d3-geo-projection/>

⁴⁵<http://localhost:8080/06-making-a-map/completed-projections>

geoMercator



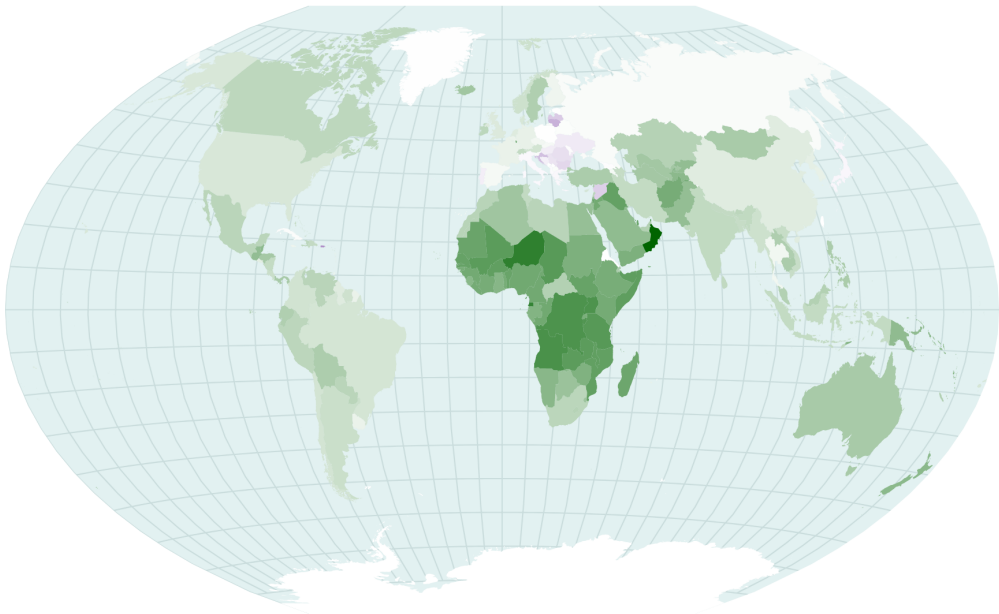
Mercator projection

Mercator has been around for a long time — it was created in 1569, before Antarctica had even been discovered! Its parallel lines preserve true angles, which made it easy for sailors to use for navigation, but it sacrifices vertical distortion for horizontal distortion.

You can see this distortion in the **graticule** (grid lines on the map) — see how tall the “squares” get near the top of the map. Also note how the country sizes get skewed as they approach the poles — Greenland is depicted as the same as Africa!

Another option is the **Winkel-Tripel** projection, which attempts to balance the three types of distortion: area, direction, and distance.

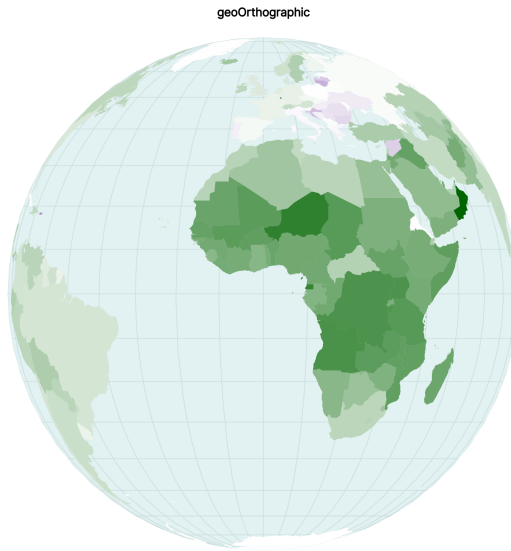
geoWinkel3



Winkel Tripel projection

Land near the North and South poles is still enlarged, but the country shapes and sizes are more accurate.

If we want to show a globe, we can use the `d3.geoOrthographic()` projection.



Orthographic projection

Note that the type of projection matters more with maps that cover more area. The more we're zoomed in, the less distortion we'll need to flatten our round Earth shape. For example, a close-up of a city will look basically the same in two different projections.

There really isn't a "right" map projection to use — they all have to distort *something*, mapping a 3D shape into 2D. As a general rule of thumb, a variant of the **Mercator**, the **Transverse Mercator** (`d3.geoTransverseMercator()`) is a good bet for showing maps that cover one country or smaller. The **Winkel Tripel** (`d3.geoWinkel3()`) or **Equal Earth** (`d3.geoEqualEarth()`) are good bets for maps covering larger areas, such as the whole world. But this is really the tip of the iceberg - if you're interested, read more about [how projections work](https://en.wikipedia.org/wiki/Map_projection)⁴⁶ and [specific projections](https://en.wikipedia.org/wiki/List_of_map_projections)⁴⁷.

Finishing creating our chart dimensions

How does this information help us decide our chart height? We're going to plot the whole Earth to cover the full bounded width, then measure the height.

⁴⁶https://en.wikipedia.org/wiki/Map_projection

⁴⁷https://en.wikipedia.org/wiki/List_of_map_projections

If you remember from before, a GeoJSON object can contain features of the following types: `Point`, `MultiPoint`, `LineString`, `MultiLineString`, `Polygon`, `MultiPolygon`, `GeometryCollection`, `Feature`, or `FeatureCollection`. But none of these cover the whole Earth! Worry not, **d3-geo**⁴⁸ adds support for a type of `"Sphere"`, which will cover the whole globe.

Let's define our globe using this mock-GeoJSON format.

code/06-making-a-map/completed/draw-map.js

```
35  const sphere = ({type: "Sphere"})
```

Each of the **projections** mentioned in the previous section (and more) are implemented in either **d3-geo** or **d3-geo-projection**. We can use these **projection functions** to convert from `[longitude, latitude]` coordinates to `[x, y]` pixel coordinates. Essentially, a **projection function** is our scale in the geographic world.

Let's create our **projection function**. We'll use `d3.geoEqualEarth()` — feel free to play around with other options once we're finished drawing our map.

code/06-making-a-map/completed/draw-map.js

```
36  const projection = d3.geoEqualEarth()
```

Each **projection function** has its own default size (think: **range**). But we want our projection to be the same width as our **bounds**.

To update our projection's width, we can use its `.fitWidth()` method, which takes two parameters:

1. the width in pixels
2. a GeoJSON object

When we call this method, our `projection` will update its size so that the GeoJSON object (2) we pass will be the specified width (1).

⁴⁸https://github.com/d3/d3-geo#_path

code/06-making-a-map/completed/draw-map.js

```

36   const projection = d3.geoEqualEarth()
37     .fitWidth(dimensions.boundedWidth, sphere)

```

Okay great, but how do we get the **height** of our **sphere**?

`d3.geoPath()`⁴⁹ is similar to the line generator (`d3.line()`) we used for our timeline in **Chapter 1**. When we pass it our projection, it will create a generator function that will help us create our geographical shapes.

code/06-making-a-map/completed/draw-map.js

```

39   const pathGenerator = d3.geoPath(projection)

```

Let's test it out, what happens when we pass it our **sphere**:

```
console.log(pathGenerator(sphere))
```

```

M137.6850612805441,2.842170943040401e-14L337.85,2.842170943040401e-14L538.0149387194559,2.842170943040401e-14L573.2969871814703,17.046318574021143L601.7388721244954,35.11582806465225L627.4033152905058,57.060601880579455L648.1636120831404,81.74126598217812L656.4778194202438,94.8373133047876L663.3459297549905,108.31468205596332L668.7284340918884,122.09019037238977L672.5945365846244,136.08545558369653L674.922602578265,150.22546379143057L675.7,164.4373301346346L674.922602578265,178.64919647783864L672.5945365846244,192.78920468557268L668.7284340918884,206.78446989687944L663.3459297549905,220.5599782133059L656.4778194202438,234.03734693879045L648.1636120831404,247.1333942870911L627.4033152905058,271.8140583886898L601.7388721244954,293.75883220461697L573.2969871814703,311.82834169524807L538.0149387194559,328.87466026926916L337.85,328.87466026926916L137.6850612805441,328.87466026926916L102.40301281852976,311.82834169524807L73.96112787550464,293.75883220461697L48.296684709494286,271.8140583886898L27.536387916859724,247.1333942870911L19.222180579756184,234.03734693879045L12.35407024500961,220.5599782133059L6.9715659081117,206.78446989687944L3.1054634153755956,192.78920468557268L0.7773974217350315,178.64919647783864L5.684341886080802e-14,164.4373301346346L0.7773974217350315,150.22546379143057L3.1054634153755956,136.08545558369653L6.9715659081117,122.09019037238977L12.35407024500961,108.31468205596332L19.222180579756184,94.8373133047876L27.536387916859724,81.74126598217812L48.296684709494286,57.060601880579455L73.96112787550464,35.11582806465225L102.40301281852976,17.046318574021143L137.6850612805441,2.842170943040401e-14Z

```

pathGenerator result

That looks familiar! Perfect, we got back a `<path> d` string. But how do we find out how **tall** that path would be?

Thankfully, our `pathGenerator()` has a `.bounds()` method that will return an array of `[x, y]` coordinates describing a bounding box of a specified GeoJSON object. Let's test that by logging the bounding box for our sphere:

⁴⁹<https://github.com/d3/d3-geo/blob/master/README.md#geoPath>

```
console.log(pathGenerator.bounds(sphere))
```

```
▼ (2) [Array(2), Array(2)] ⓘ
  ► 0: (2) [0, 0]
  ► 1: (2) [831.4, 404.656493337088]
    sphere bounds
```

Great! When drawn on our map, the whole Earth will span from 0px to 831.4px horizontally and from 0px to 404.7px vertically. We can use [ES6 array destructuring](#)⁵⁰ to grab the y1 value.

code/06-making-a-map/completed/draw-map.js

```
39  const pathGenerator = d3.geoPath(projection)
40  const [[x0, y0], [x1, y1]] = pathGenerator.bounds(sphere)
```

We want the entire Earth to fit within our **bounds**, so we'll want to set our `boundedHeight` to *just* cover our sphere.

code/06-making-a-map/completed/draw-map.js

```
42  dimensions.boundedHeight = y1
43  dimensions.height = dimensions.boundedHeight
44    + dimensions.margin.top
45    + dimensions.margin.bottom
```

Draw canvas

Now we're back in familiar territory! Let's create our **wrapper** and **bounds** elements. See if you can do this step from memory.

First, we want to add a new `<svg>` element (our **wrapper**) to the existing `<div>` with an `id` of "wrapper". Then we want to add a `<g>` element (our **bounds**) and shift it to respect our `top` and `left` margins.

⁵⁰https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment#Array_destructuring

code/06-making-a-map/completed/draw-map.js

```
49  const wrapper = d3.select("#wrapper")
50    .append("svg")
51      .attr("width", dimensions.width)
52      .attr("height", dimensions.height)
53
54  const bounds = wrapper.append("g")
55    .style("transform", `translate(${
56      dimensions.margin.left
57    }px, ${
58      dimensions.margin.top
59    }px)`)
```

Create scales

Next up, we need to create our scales. Typically, we would create **x** and **y** scales, but those are covered by our **projection**. We'll only need a scale to help us turn our **metric values** (population growth amounts) into **color values**.

Remember our object of **country id** and **population growth values**? We can grab all of the population growth values by using `Object.values()`, which returns an object's values in an array. More info [here](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_objects/Object/values)⁵¹, for the curious.

code/06-making-a-map/completed/draw-map.js

```
63  const metricValues = Object.values(metricDataByCountry)
```

Then we can extract the smallest and largest values by using `d3.extent()`.

⁵¹https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_objects/Object/values

code/06-making-a-map/completed/draw-map.js

64

```
const metricValueExtent = d3.extent(metricValues)
```

Let's take a look at the range we're dealing with with `console.log(metricValueExtent)`

```
► (2) [-2.05660023263167, 4.669194641437]
```

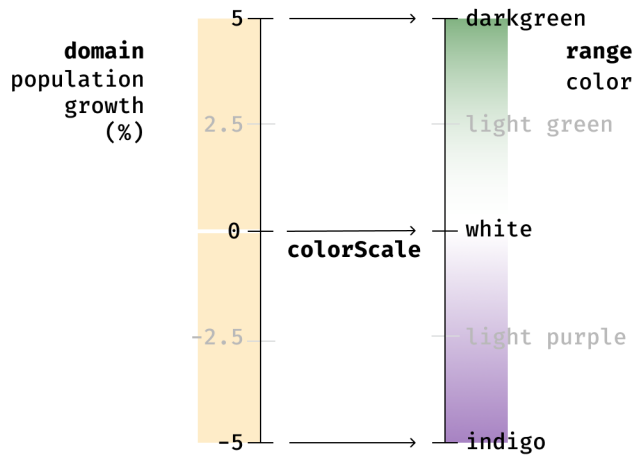
Metric values extent

Aha! Our extent starts below zero, meaning that some countries had a *negative* population growth in 2017. Let's represent population declines on a **red** color scale and population growths on a **green** color scale.

This kind of color scale is called a **diverging** color scale, which we'll cover in more detail in **Chapter 7**.

So far, we've only create scales that have one *minimum* and one *maximum* value for both their **domain** and **range**. But we can create *piecewise* scales, which are basically multiple scales in one. If we use a **domain** and **range** with more than two values, d3 will create a scale with more than two "anchors".

In this case, we want to create a scale with a "middle" population growth of 0% that converts to a middle color of **white**. Population growth amounts above 0 should be converted with a color scale from **white** (0) to **green** (5) and population growth amounts below 0 should be converted with a color scale from **red** (-2) to **white** (0).



Color scale

Creating a *piecewise* scale like this is easier than it might seem: just add a middle value to both the **domain** and the **range**.

code/06-making-a-map/completed/draw-map.js

```

65  const maxChange = d3.max([-metricValueExtent[0], metricValueExtent[1]\
66  ])
67  const colorScale = d3.scaleLinear()
68    .domain([-maxChange, 0, maxChange])
69    .range(["indigo", "white", "darkgreen"])

```

We have a choice here:

1. create a scale which scales evenly on both sides (eg. `.domain([-5, 0, 5])`)

This approach will let the viewer see if countries are shrinking at the same pace as countries that are growing, since a light red (-2.5%) will correlate to a light green (2.5%).

2. create a scale which scales unevenly from white to red and white to green (eg. `.domain([-2, 0, 5])`)

This approach will maximize the color scale, making distinctions between countries with different growth rates clear. However, it can be confusing when a light red color (-0.1%) doesn't match the scale of a light green color (2.5%).

We'll go with approach number 1 here, to keep things simple, but play around with the two approaches yourself to get a feel for the differences! How might the legend (coming up) look different with approach number 2?

Why did we choose **indigo** and **darkgreen** as our colors? A more semantic mapping to “negative” and “positive” might be **red** and **green**, but the most common form of color blindness is red-green color blindness, and we want our readers to be able to see differences in our color scale. We'll talk more about this in **Chapter 7**.

Draw data

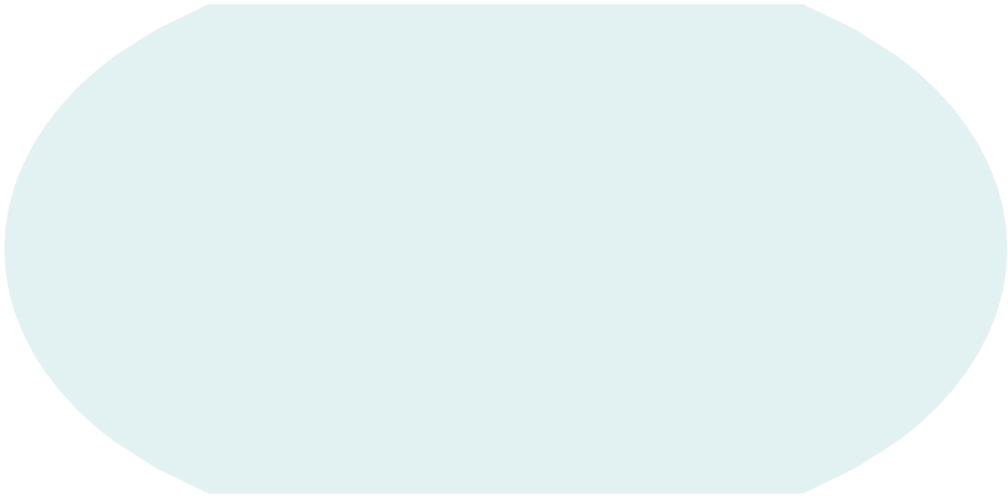
Here comes the action! To start, we'll draw our Earth outline. It makes sense to draw this first, since SVG elements layer based on their order in the DOM, and we'll want all other elements to cover this outline.

Remember that our `pathGenerator` is aware of the **projection** we're using and will act as a **scale**, converting a GeoJSON object into a `<path>` **d** string.

`code/06-making-a-map/completed/draw-map.js`

```
72  const earth = bounds.append("path")
73    .attr("class", "earth")
74    .attr("d", pathGenerator(sphere))
```

Great! Now we can see the outline of our earth.



Earth outline

Note that we're styling our elements in the `/code/06-making-a-map/styles.css` file, using their class names. To see the elements without the styling, remove the class attribute.

Next, let's display a **graticule** on our map. A **graticule** is a grid of latitudinal and longitudinal lines — essentially tick marks for a map. These are helpful for aligning map elements that are far apart, and also for conveying how the projection is distorting the globe.

Thankfully, we can easily create a GeoJSON graticule — `d3.geoGraticule10()` will generate a classic graticule with lines every 10 degrees.

`code/06-making-a-map/completed/draw-map.js`

76

```
const graticuleJson = d3.geoGraticule10()
```

There are many ways to configure this GeoJSON graticule, check out the options in the [d3-geo docs](https://d3js.org/d3-geo/docs#graticule)^a.

^a<https://github.com/d3/d3-geo#geoGraticule>

If we log this graticule object to the console (`console.log(graticuleJson)`), we can see that it's a GeoJSON object of type "MultiLineString".

```
▼ {type: "MultiLineString", coordinates: Array(53)} ⓘ
  ► coordinates: (53) [Array(3), Array(3), Array(3), Array(3), Array(145), Array(3), Array(3), Ar...
    type: "MultiLineString"
    ► __proto__: Object
```

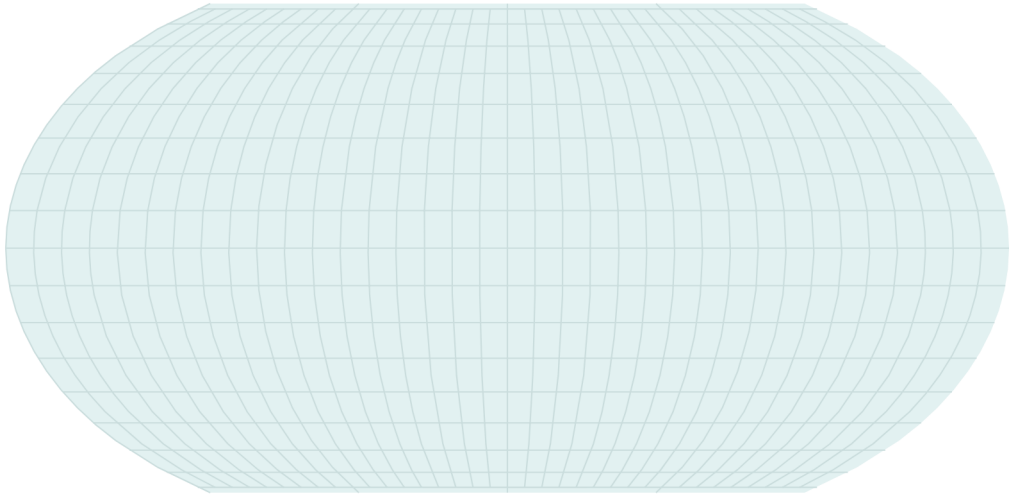
graticule.json

Thankfully, our `pathGenerator()` knows how to handle any GeoJSON type — let's use it to draw our graticules.

code/06-making-a-map/completed/draw-map.js

```
77  const graticule = bounds.append("path")
78    .attr("class", "graticule")
79    .attr("d", pathGenerator(graticuleJson))
```

Wonderful!



globe with graticule

Finally, let's draw our countries. We'll use the **data join** method we used in **Chapter 2**. First, we'll select all of the elements with a class of `.country`. Remember: even though these elements don't yet exist, this is priming our **d3 selection object** to bind data to similar elements.

code/06-making-a-map/completed/draw-map.js

```
81  const countries = bounds.selectAll(".country")
82    .data(countryShapes.features)
```

Next, we'll bind our data to our selection object. Remember what our `countryShapes` GeoJSON object looks like?

```

▼ {type: "FeatureCollection", name: "ne_50m_admin_0_countries", crs: {...}, features: Array(241)} ⓘ
  ► crs: {type: "name", properties: {...}}
  ▼ features: Array(241)
    ▼ [0 ... 99]
      ► 0: {type: "Feature", properties: {...}, geometry: {...}}
        ▼ 1:
          ► geometry: {type: "Polygon", coordinates: Array(1)}
          ► properties: {featurecla: "Admin-0 country", scalerank: 1, LABELRANK: 3, SOVEREIGNT: "Zambia", S...
            type: "Feature"
          ► __proto__: Object
        ► 2: {type: "Feature", properties: {...}, geometry: {...}}
        ► 3: {type: "Feature", properties: {...}, geometry: {...}}
        ► 4: {type: "Feature", properties: {...}, geometry: {...}}
        ► 5: {type: "Feature", properties: {...}, geometry: {...}}
        ► 6: {type: "Feature", properties: {...}, geometry: {...}}
        ...
        ► 7: {type: "Feature", properties: {...}, geometry: {...}}

```

GeoJSON data, features

Because d3 will create **one element per item in the dataset** we pass to `.data()`, we'll want to use the list of features instead of the whole object.

code/06-making-a-map/completed/draw-map.js

```

81  const countries = bounds.selectAll(".country")
82    .data(countryShapes.features)
83    .enter().append("path")

```

Next, we'll use `.enter().append("path")` to tell d3 to create one new `<path>` for each item in our list of countries.

For a reminder of how `.enter().append()` works, refer back to **Chapter 3**.

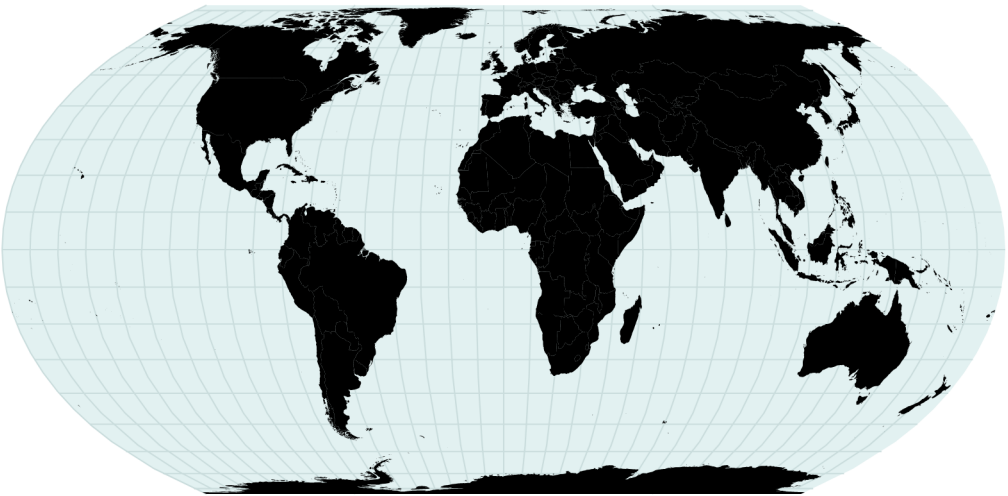
We'll give each of these `<path>`s a class of "country" and then pass it straight to our `pathGenerator()` to get a **d** string, generating the shape of each country.

code/06-making-a-map/completed/draw-map.js

```
81  const countries = bounds.selectAll(".country")
82    .data(countryShapes.features)
83    .enter().append("path")
84      .attr("class", "country")
85      .attr("d", pathGenerator)
86      .attr("fill", d => {
```

Remember that `.attr("d", pathGenerator)` is the same as `.attr("d", d => pathGenerator(d))`.

If we look at our browser now, we'll see all of our countries!



globe with countries

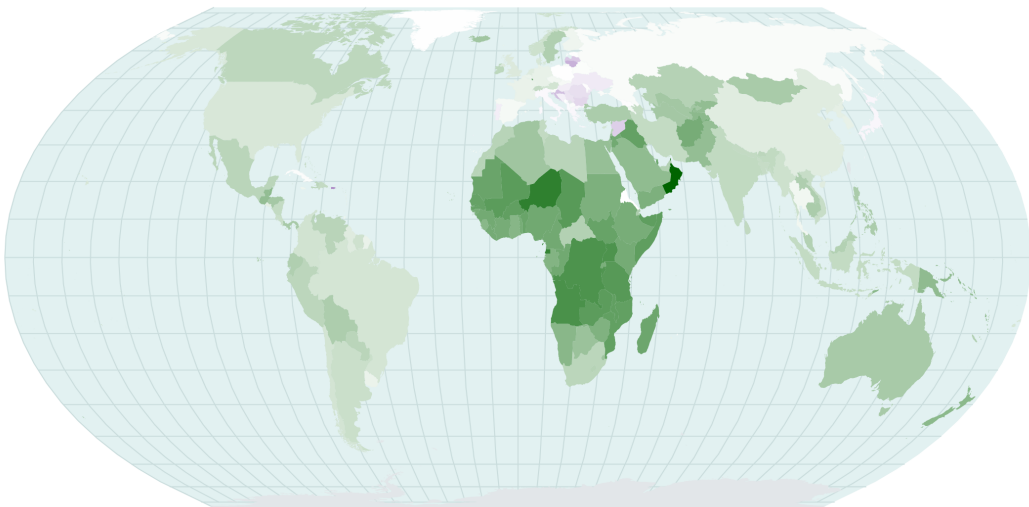
We have one last task — we want to color our countries based on their population growth amount. We'll need to set their fill by looking up the country's id in the `metricDataByCountry` object we created at the top of our file, then passing that id to our `colorScale()`.

Since some countries might be missing from our dataset, we'll want to indicate that on our map by coloring them white.

code/06-making-a-map/completed/draw-map.js

```
87     const metricValue = metricDataByCountry[countryIdAccessor(d)]
88     if (typeof metricValue == "undefined") return "#e2e6e9"
89     return colorScale(metricValue)
```

Perfect! Now we can see our map in full color!



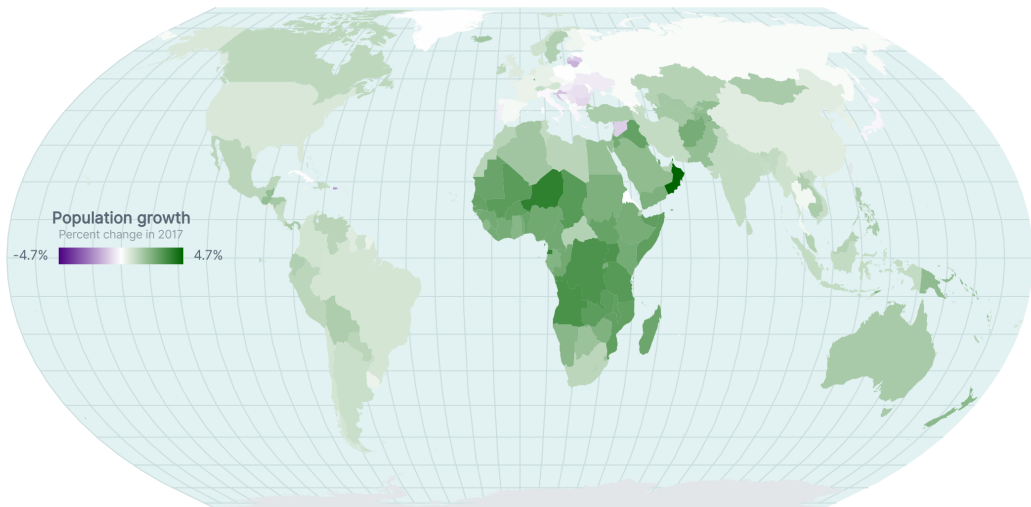
globe with colored countries

It's clear now that the highest growth countries are in Africa and the Middle East, and that a cluster of European countries shrunk in 2017. Try to find a few other insights that are easy to see now that we've visualized this data with a map.

Draw peripherals

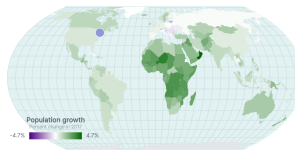
Drawing a legend

Data visualizations should be able to stand on their own, without accompanying text. Let's create a legend to explain our color scale.

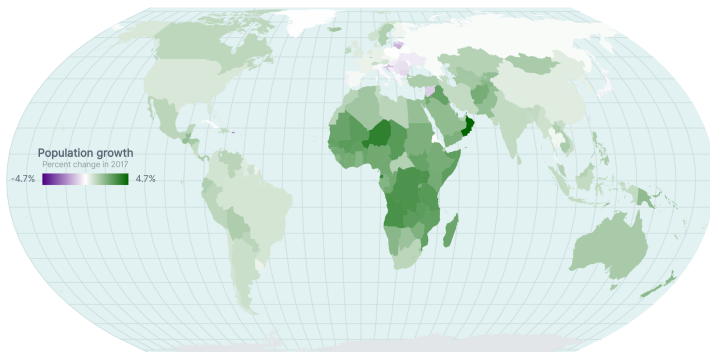


globe with legend

To start, we'll create a `<g>` element and position it on the left side of our map. To get a taste of making our map work at different screen sizes, we'll position it differently based on the screen size. On a skinnier screen, let's put it at the bottom of the map, and on larger screens, we can place it halfway down the map (to the left of the Americas).



globe with legend, small



globe with legend, large

code/06-making-a-map/completed/draw-map.js

```

94  const legendGroup = wrapper.append("g")
95    .attr("transform", `translate(${
96      120
97    }, ${
98      dimensions.width < 800
99      ? dimensions.boundedHeight - 30
100      : dimensions.boundedHeight * 0.5
101    })`)

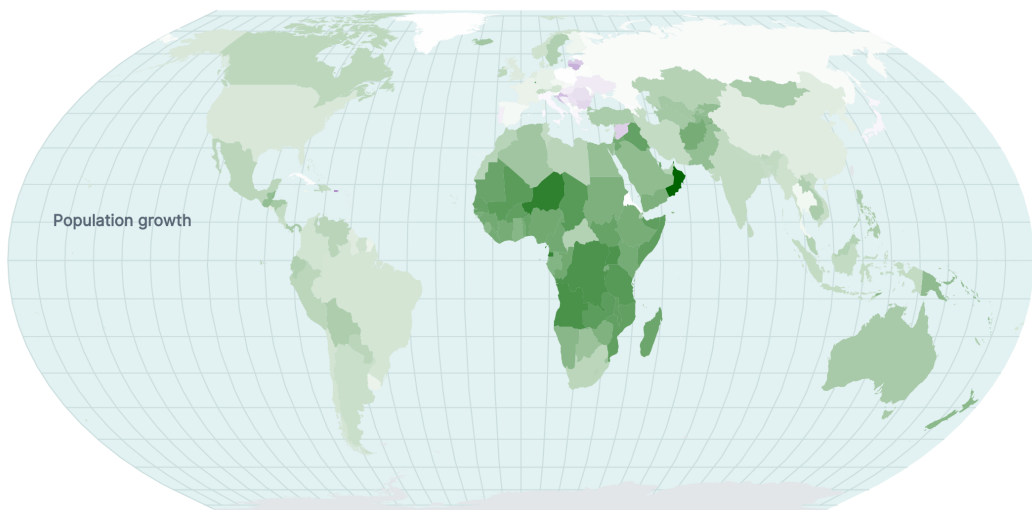
```

To keep things simple, let's align the top of our `<g>` element to the top of the gradient bar. When we add our legend title, we'll move it up 23 pixels to give the bar some space.

code/06-making-a-map/completed/draw-map.js

```
103  const legendTitle = legendGroup.append("text")
104      .attr("y", -23)
105      .attr("class", "legend-title")
106      .text("Population growth")
```

Great! Now we can see where our legend group is positioned.



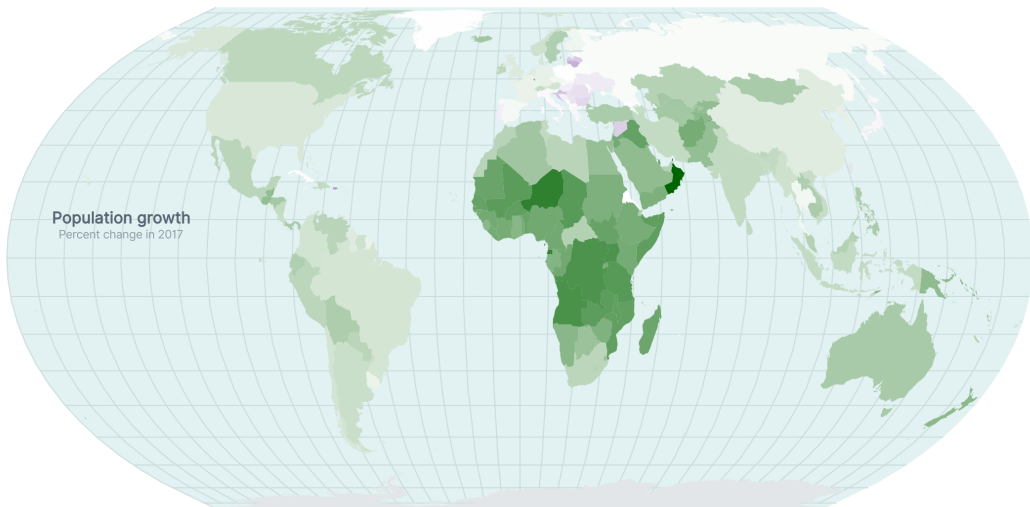
globe with legend title

Next, let's add a byline to give the reader more context.

code/06-making-a-map/completed/draw-map.js

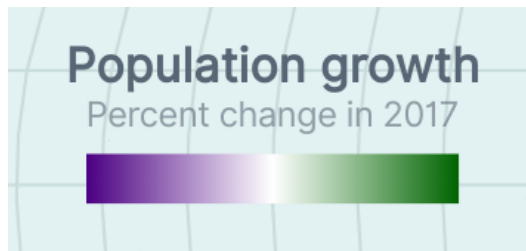
```
108  const legendByline = legendGroup.append("text")
109      .attr("y", -9)
110      .attr("class", "legend-byline")
111      .text("Percent change in 2017")
```

Note that we're adding classes that match up with existing CSS styles — feel free to check out `styles.css` to investigate. We're using the CSS property `text-anchor: middle` to horizontally center our `<text>`.



globe with legend byline

Next up, we'll create the bar showing the colors in our scale.



legend gradient bar

Remember when we created a `<defs>` SVG element for our `<clipPath>` in **Chapter 4**? Elements created within `<defs>` won't be visible by themselves, but we can use them later. Let's create a `<defs>` element to store a gradient.

code/06-making-a-map/completed/draw-map.js

```
113  const defs = wrapper.append("defs")
```

Since we'll be creating a bar here and referencing it in another place, let's define a variable to hold the `id` of our gradient.

code/06-making-a-map/completed/draw-map.js

```
114  const legendGradientId = "legend-gradient"
```

Time to create our gradient! To make a gradient in SVG, we'll need to create a `<linearGradient>`⁵² SVG element. Within that, we'll create several `<stop>` SVG elements that will tell the gradient what colors to interpolate between, using `stop-color` and `offset` attributes.

For example, this HTML code:

```
<linearGradient>  
  <stop stop-color="#12CBC4" offset="0%"></stop>  
  <stop stop-color="#FFC312" offset="50%"></stop>  
  <stop stop-color="#B53471" offset="100%"></stop>  
</linearGradient>
```

will create this gradient:



Gradient example

The `offset`'s percentage value is *in proportion to the element using the* `<linearGradient>`.

Let's try it out!

⁵²<https://developer.mozilla.org/en-US/docs/Web/SVG/Element/linearGradient>

We'll set the `id` attribute to our `legendGradientId` so that we can reference our gradient later.

code/06-making-a-map/completed/draw-map.js

```
115   const gradient = defs.append("linearGradient")
116       .attr("id", legendGradientId)
```

To add our stops, we'll use `colorScale.range()` to get an array of the colors in our color scale. Then we'll use `.enter().append()` to create one stop per color. We'll use the color's index to calculate the percent offset (from 0% to 100%).

code/06-making-a-map/completed/draw-map.js

```
115   const gradient = defs.append("linearGradient")
116       .attr("id", legendGradientId)
117       .selectAll("stop")
118       .data(colorScale.range())
119       .enter().append("stop")
120       .attr("stop-color", d => d)
121       .attr("offset", (d, i) => `${
122         i * 100 / 2 // 2 is one less than our array's length
123       }%`)
```

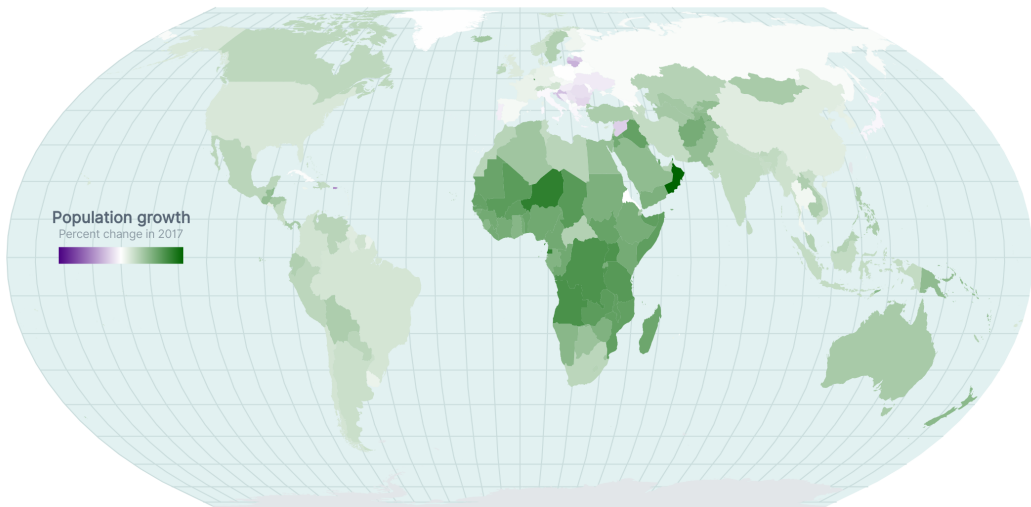
To use a `<linearGradient>`, all we need to do is set the `fill` or `stroke` of a SVG element to `url(#GRADIENT_ID)` (where `GRADIENT_ID` matches the `<linearGradient>`'s `id` attribute).

Let's create a `<rect>` that's centered horizontally and is filled with our gradient's `id`.

code/06-making-a-map/completed/draw-map.js

```
125  const legendWidth = 120
126  const legendHeight = 16
127  const legendGradient = legendGroup.append("rect")
128    .attr("x", -legendWidth / 2)
129    .attr("height", legendHeight)
130    .attr("width", legendWidth)
131    .style("fill", `url(#${legendGradientId})`)
```

Wonderful! Now we can see our color scale in our legend. Note how it highlights that most countries are *growing* and the countries that are shrinking are only shrinking a small amount.



globe with legend

Lastly, let's label our color scale with the minimum and maximum percent change in our scale. We'll start by labeling the right side, positioning our text 10 pixels to the right of our bar.

code/06-making-a-map/completed/draw-map.js

```
133     const legendValueRight = legendGroup.append("text")
134         .attr("class", "legend-value")
135         .attr("x", legendWidth / 2 + 10)
136         .attr("y", legendHeight / 2)
137         .text(`$${d3.format(".1f")(maxChange)}%`)
```

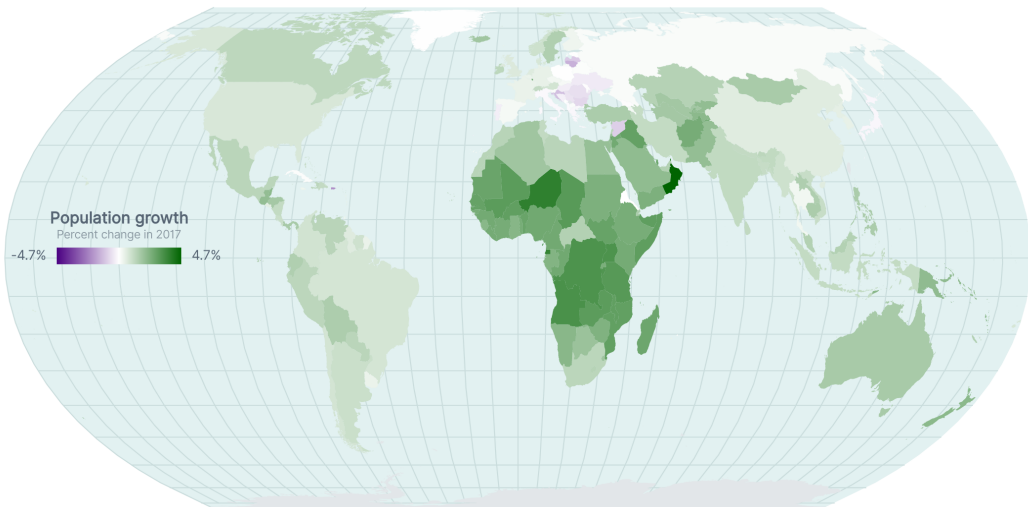
If you're wondering how our text is *vertically centered* with the gradient bar, check out the styles.css file. We're using the `dominant-baseline: middle` CSS property.

And now we'll label the left side — we can use the `text-anchor: end` CSS property to align the right side of our text to the left of our bar.

code/06-making-a-map/completed/draw-map.js

```
139     const legendValueLeft = legendGroup.append("text")
140         .attr("class", "legend-value")
141         .attr("x", -legendWidth / 2 - 10)
142         .attr("y", legendHeight / 2)
143         .text(`$${d3.format(".1f")(-maxChange)}%`)
144         .style("text-anchor", "end")
```

Great! Now readers will be able to tell what our colors mean, and also the largest decline and increase in percent of population.



globe with legend

We have a choice here on whether or not we want to use `.nice()` on our color scale to have friendlier min and max values. A color scale that spans -5% to 5% is easier to reason about, but there are two downsides in this case:

1. With a color scale, it's nice (no pun intended) to easily see the max change, which are labeled on our legend.
2. Since it's so hard to distinguish between colors, squishing the scale will make differences harder to see.

As with many aspects of data visualizations, there is no one right answer. But it's important to know the pros and cons and decide based on your goals.

Want more practice creating color scale legends? Adding a legend to the scatter plot we made in **Chapter 2** could help solidify the steps!

Marking my location

One of the fun parts of building in the browser is that we can interact with the user. Let’s take advantage of this by plotting the reader’s location on our map.

Modern browsers have a global `navigator` object, which provides information about the device accessing the website.

													
	 Chrome	 Edge	 Firefox	 Internet Explorer	 Opera	 Safari	 Android webview	 Chrome for Android	 Edge Mobile	 Firefox for Android	 Opera for Android	 Safari on iOS	 Samsung Internet
Navigator	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes

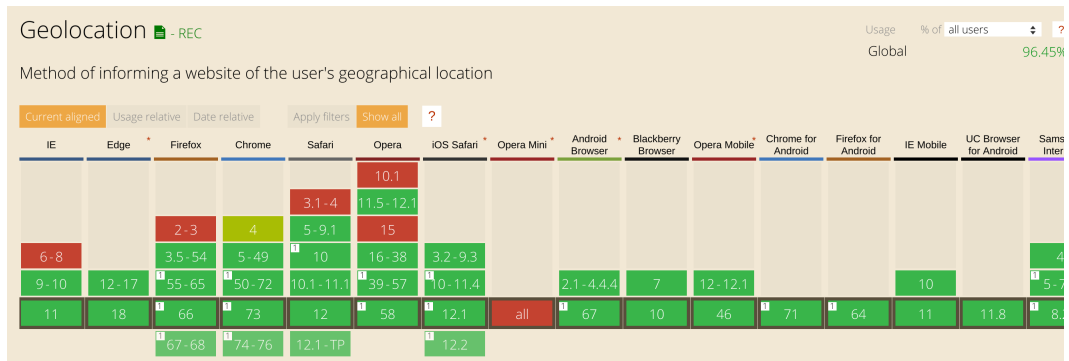
`navigator` support, from developer.mozilla.org

[Here’s a good resource^a](#) if you’re curious about present support or available keys.

^a<https://developer.mozilla.org/en-US/docs/Web/API/Navigator>

`navigator.geolocation` has a method for grabbing the browser’s location:
`.getCurrentPosition()`. [Most modern browsers⁵³](#) have support for this method.

⁵³<https://caniuse.com/#feat=geolocation>

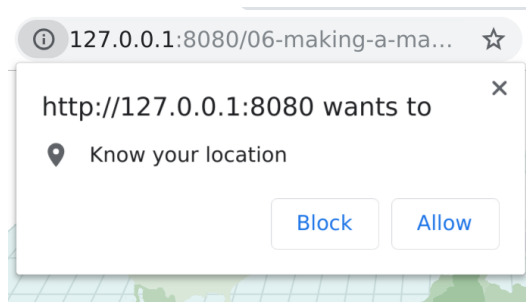


geolocation support, from caniuse.com

We can pass `.getCurrentPosition()` a callback function that has the user's location. Let's run this at the bottom of our code and log out the passed information.

```
navigator.geolocation.getCurrentPosition(myPosition => {
  console.log(myPosition)
})
```

The browser will ask to access our location the first time we load the page. The callback will only run if we click **Allow**.



Browser - dialog to receive location

We can see the structure of `myLocation` after the callback runs.

```

▼ Position {coords: Coordinates, timestamp: 1553996869267} ⓘ
  ▼ coords: Coordinates
    accuracy: 1168
    altitude: null
    altitudeAccuracy: null
    heading: null
    latitude: 43.1
    longitude: -77.6
    speed: null
    ► __proto__: Coordinates
  timestamp: 1553996869267
  ► __proto__: Position

```

myPosition

Great! We have access to both a latitude and a longitude — we know how to handle that! Let's use our projection to turn our location (in a [latitude, longitude] array), into a set of [x, y] coordinates.

code/06-making-a-map/completed/draw-map.js

```

147     const [x, y] = projection([
148         myPosition.coords.longitude,
149         myPosition.coords.latitude
150     ])

```

Next, we'll draw a <circle> with a class of "my-location" in that location. We'll also animate the circle's radius from 0 to 10, to draw the eye.

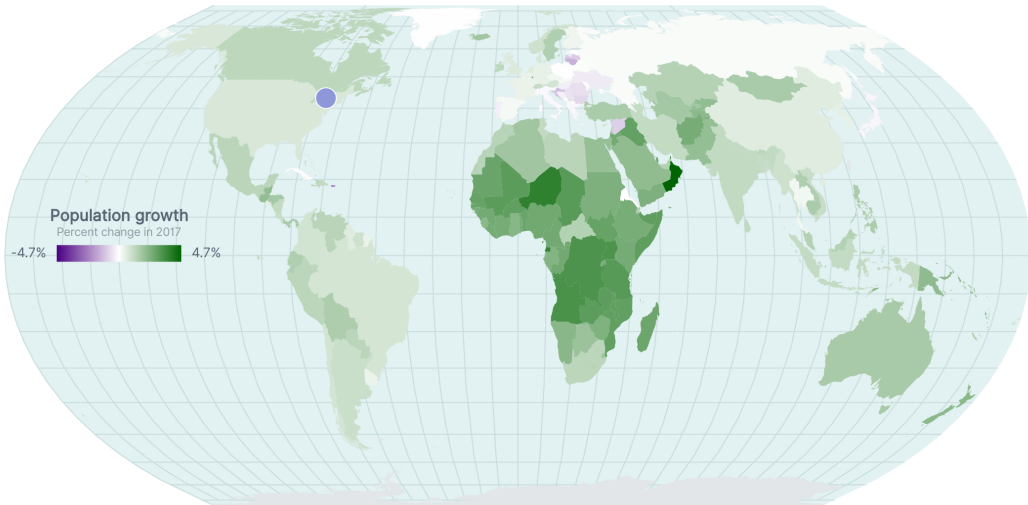
code/06-making-a-map/completed/draw-map.js

```

146     navigator.geolocation.getCurrentPosition(myPosition => {
147         const [x, y] = projection([
148             myPosition.coords.longitude,
149             myPosition.coords.latitude
150         ])
151         const myLocation = bounds.append("circle")
152             .attr("class", "my-location")
153             .attr("cx", x)
154             .attr("cy", y)
155             .attr("r", 0)
156             .transition().duration(500)
157             .attr("r", 10)
158     })

```

Nice! It's good to keep in mind that users relate to our data visualizations through their personal lens — seeing themselves in a visualization helps give it meaning.



map with my location

Set up interactions

Lastly, let's use what we learned in the last chapter to give the user more information when they hover over a country.

First, we want to initialize our **mouse events**, using the `.on()` method for our `countries` elements.

code/06-making-a-map/completed/draw-map.js

```
162 countries.on("mouseenter", onMouseEnter)
163           .on("mouseleave", onMouseLeave)
```

Next, we want to grab the tooltip element — this is already created in our `index.html` file. Having a static `tooltip` reference outside of our mouse event callbacks keeps us from having to look it up every time a country is hovered.

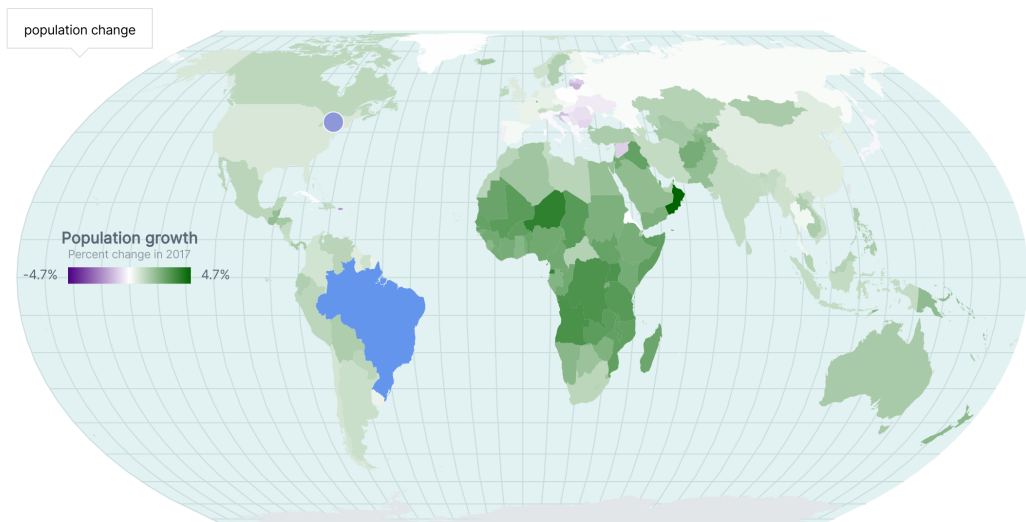
code/06-making-a-map/completed/draw-map.js

```
165  const tooltip = d3.select("#tooltip")
```

Next, let's initialize our `onmouseenter()` and `onmouseleave()` callback functions and have them toggle our tooltip's visibility.

```
function onmouseenter(datum) {  
  tooltip.style("opacity", 1)  
}  
  
function onmouseleave() {  
  tooltip.style("opacity", 0)  
}
```

Great, now our tooltip shows up when we hover over a country.



map on hover

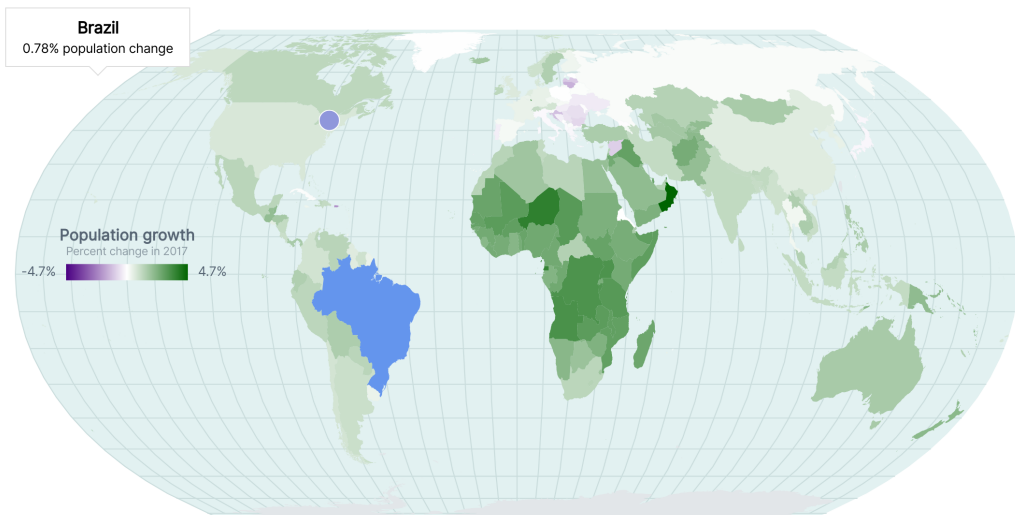
Next, let's populate the text of our tooltip. Remember that the first parameter of our mouse event (that we've named `datum`) contains the data bound to the hovered element. We can use this to grab the country name and population growth value. We

can check the `index.html` file to find the ids of the elements which will contain this text.

code/06-making-a-map/completed/draw-map.js

```
166 function onMouseEnter(datum) {  
167   tooltip.style("opacity", 1)  
168  
169   const metricValue = metricDataByCountry[countryIdAccessor(datum)]  
170  
171   tooltip.select("#country")  
172     .text(countryNameAccessor(datum))  
173  
174   tooltip.select("#value")  
175     .text(`${d3.format(",.2f")(metricValue || 0)}%`)
```

Perfect! Now our tooltip updates to give us information about the currently hovered country.



map on hover, with tooltip text

But how do we know where to position the tooltip? Our `pathGenerator` comes to the rescue again!

Remember how we used `pathGenerator.bounds(sphere)` to get the height of the entire globe? Our `pathGenerator` also has a `.centroid()` method that will give us the center of the passed GeoJSON object. Let's see what we get when we ask for the center of our hovered country's bound data:

```
console.log(pathGenerator.centroid(datum))
```

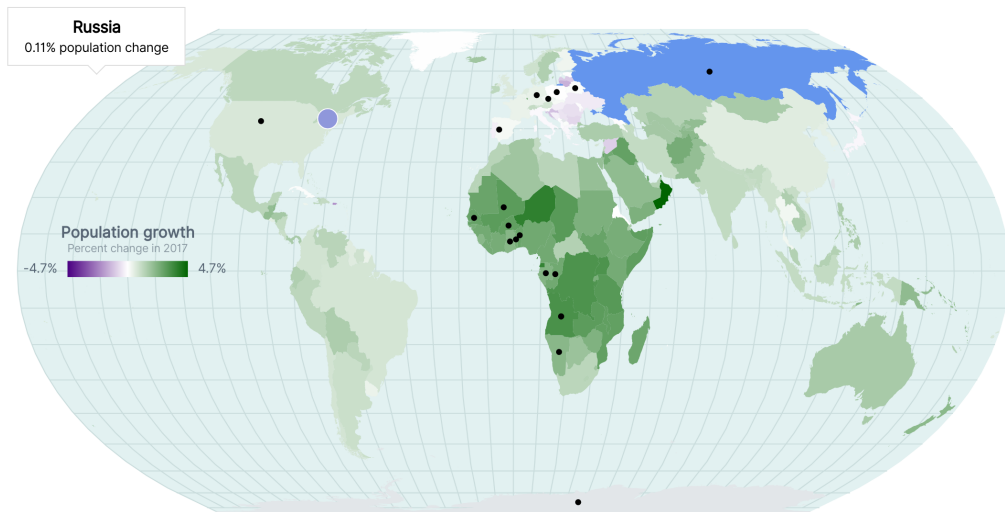
```
▶ (2) [362.60341787403684, 289.20854414569305]
```

map on hover, `.centroid()` response

Great! This looks like an `[x, y]` coordinate. Let's grab that `x` and `y` coordinate and position a new `<circle>` element to make sure this is the correct coordinate.

```
function onMouseEnter(datum) {  
  // ...  
  const [centerX, centerY] = pathGenerator.centroid(datum)  
  const hoveredCircle = bounds.append("circle")  
    .attr("cx", centerX)  
    .attr("cy", centerY)  
    .attr("r", 3)  
}
```

When we hover a country, we can see that a dot appears exactly in the center.



map on hover, dots at `.centroid()`

Let's take out that `hoveredCircle` code and instead position our tooltip. Remember that we want to shift it with our `top` and `left` margins, since our tooltip element lives outside of our **bounds**. Just like in the last chapter, we'll need to use `calc()` to accomodate the tooltip's own width and height.

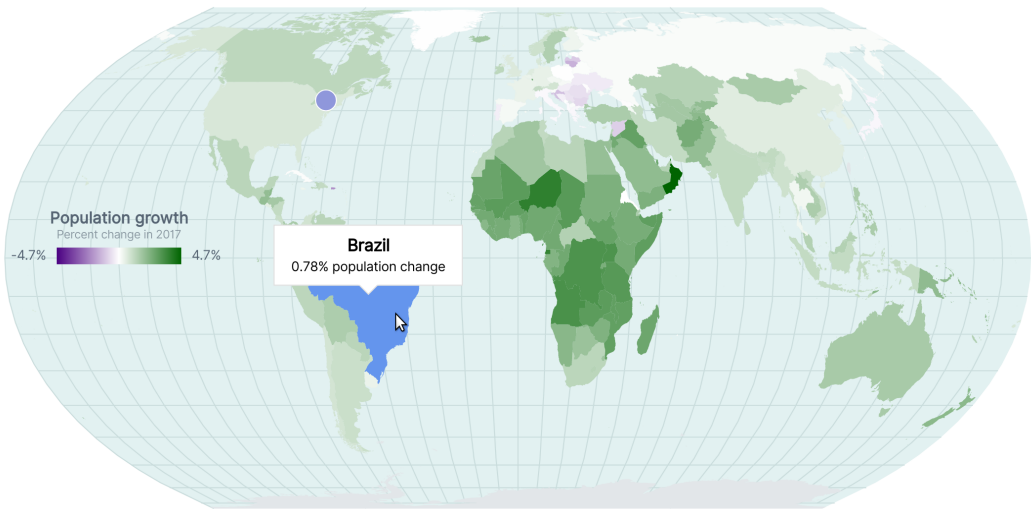
code/06-making-a-map/completed/draw-map.js

```

177     const [centerX, centerY] = pathGenerator.centroid(datum)
178
179     const x = centerX + dimensions.margin.left
180     const y = centerY + dimensions.margin.top
181
182     tooltip.style("transform", `translate(`
183       + `calc( -50% + ${x}px),`
184       + `calc(-100% + ${y}px)`
185       + `)``

```

Perfect! Now our map is easy to explore, and users can dig into the exact population growth numbers.



map on hover, with correctly positioned tooltip

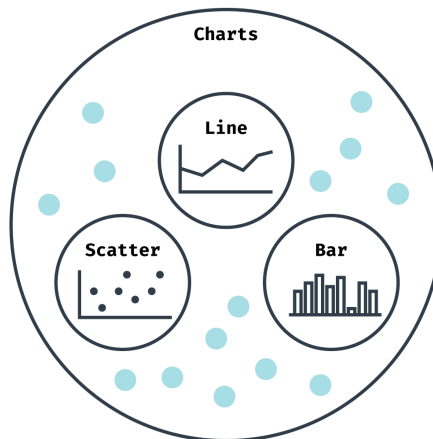
With this method of positioning the tooltip, it can be very hard to get more information about small countries. An alternative is to use the **voronoi** method we used for our scatter plot in the last chapter. This is implemented in the `/code/06-making-a-map/completed-voronoi/` folder. Feel free to check out the code and compare the two methods in the browser.

Wrapping up

Way to go! We learned a *ton* of new concepts in this chapter. Play around with downloading different GeoJSON files, changing the projection, and using different data to build on the skills we learned in here. Creating maps is a whole discipline, and this is just the tip of the iceberg!

Data Visualization Basics

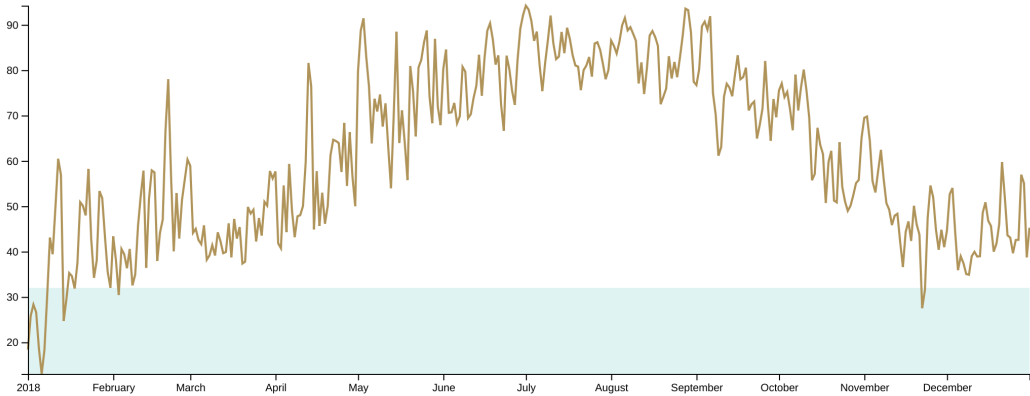
Now that we're comfortable with *how* to create a chart, we should zoom out a bit and talk about *what* chart to create. So far, we've created a line, scatter, and bar chart, but there are many more basic types to choose from, as well as charts that don't fall into a simple category.



Zooming out

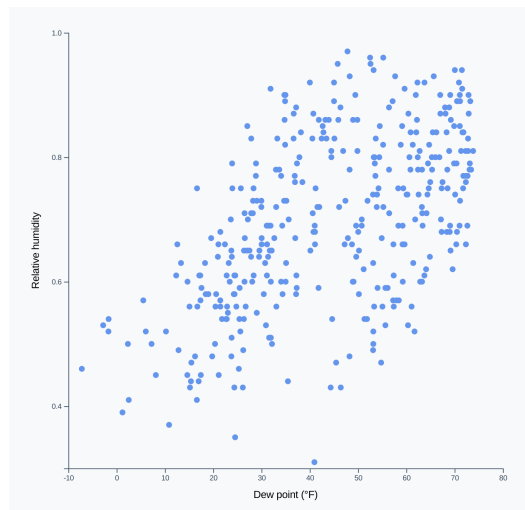
The format of our chart is the largest factor in what information our users take away — even given the same dataset! Let's look at the charts we've made with our weather data.

In **Chapter 1**, we charted a timeline of maximum temperatures. Looking at this chart, we could see how the temperature changed over time - how consistent was the weather day to day or season to season?



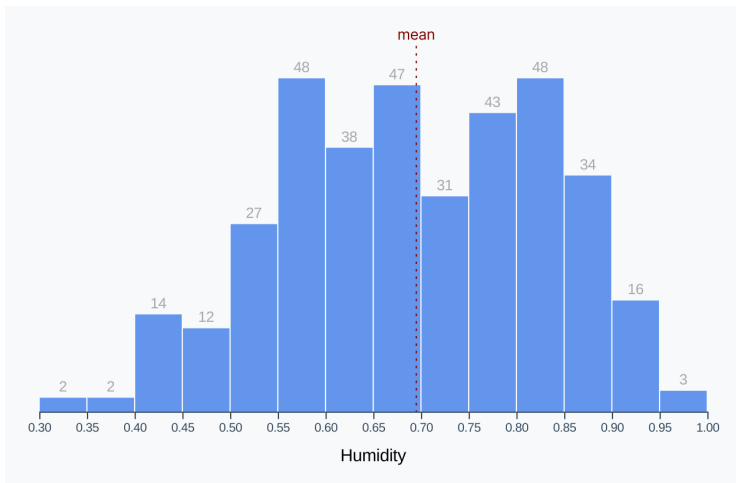
Finished line graph

In **Chapter 2**, we created a scatter plot with two metrics. Looking at this chart, we could see at how humidity and dew point are related, answering questions such as: does a high humidity also meant a high dew point?



Our Finished Scatterplot

In **Chapter 3**, we created a histogram of humidity values. Looking at this chart, we could see how much variety there was in a single metric — do most days stay around the same humidity level, or are they all very different?



Finished humidity histogram

Even with these three examples and a limited dataset, we can see how the type of chart will enable the user to answer very different questions. There are also many types of charts — thus, answering the question **What type of chart is best?** is both important and overwhelmingly open-ended. Don't worry, though — this chapter will equip you with the tools to make that decision, and quickly!

Let's start at the beginning: with our data.

Types of data

Given a dataset, the first task is to determine the structure of the available metrics. Let's look at the first item of our weather dataset.